



UTEC Posgrado

CURSO

INGENIERÍA DE DATOS

Mg. Angel Tintaya
Senior Data Engineer



The image features a dark blue background with a glowing network graph on the left side, consisting of white dots connected by thin white lines. A hand is visible at the bottom, holding a bright, glowing blue beam of light that extends horizontally across the frame. Overlaid on this background is the text 'SOLUCIONES AVANZADAS Y CASOS PRÁCTICOS' in a large, white, sans-serif font, centered in the upper half of the image.

SOLUCIONES AVANZADAS Y CASOS PRÁCTICOS

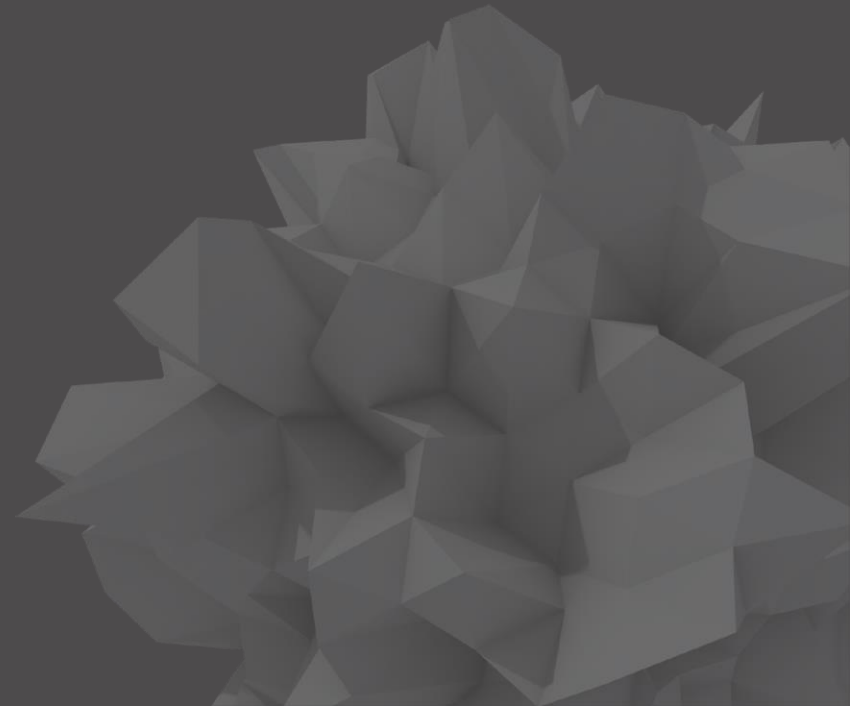


ADOPCIÓN DE DATAMESH CON MESH MEDALLION ARCHITECTURE



AGENDA DE HOY

- The road so far
- Principios del datamesh
- Caso: RetailNova
- Adopción progresiva de datamesh II
- Takeaways





The road so far



- Implementación de soluciones innovadoras para problemas complejos.
- **Optimización de Pipelines para escalabilidad y eficiencia.**
- Análisis de casos reales en sectores clave





Adopción Progresiva de Datamesh II

Objetivo

Adoptar Data Mesh de manera progresiva, empezando por dominios estratégicos, resolviendo problemas concretos del negocio mediante Data Products, y sentando las bases para escalar.



ETAPAS

ETAPA 0: Definición estratégica y gobernanza inicial

ETAPA 1:

Descubrimiento y
definición de data
products

ETAPA 2:

Infraestructura y
configuración base

ETAPA 3:

Ingesta y
almacenamiento

ETAPA 4:

Procesamiento,
calidad y
transformación

ETAPA 5:

Exposición y
gobernanza

ETAPA 6:

Escalado y
federación

ETAPA 4: Procesamiento, calidad y transformación

Objetivo: Aplicar transformaciones y asegurar calidad desde bronze → silver → gold.

Pasos	Descripción	Ejemplo
Desarrollo en Silver	Limpieza, joins, normalización, registros válidos	Realizar joins con tablas provenientes de otro data product.
Desarrollo en Gold	Agregaciones, métricas, modelos analíticos, ready-for-consumption	Crear una vista resumen del perfil del cliente con métricas clave.
Procesamiento incremental	Garantizar procesamiento incremental cuando sea necesario	Aplicar un merge incremental basado en el campo INGESTION_DATE.
Aplicar validaciones y tests de calidad	Comprobar integridad, valores esperados, duplicados.	Calcular % de registros nulos, detectar clientes sin edad o con emails inválidos.
Programar workflows o pipelines	Configurar en Databricks Job Workflows.	Workflow g0_mkt_clientes que actualiza diariamente información de clientes.
Guardar logs y monitoreo del pipeline	Registrar tiempos, errores, métricas.	Visualizar el historial de ejecuciones y el estado de cada notebook en el DAG del Job.

ETAPA 4: Desarrollo en Silver

Catálogo: Ventas

```
%sql
CREATE OR REPLACE VIEW g0_cmc_ventas.silver.vw_ventas
AS
SELECT
    venta_id,
    cliente_id,
    producto_id,
    fecha_venta,
    cantidad,
    monto_total,
    lower(canal) AS canal,
    creado,
    inserted_at
FROM g0_cmc_ventas.bronze.raw_ventas_ecommerce
UNION ALL
SELECT
    venta_id,
    cliente_id,
    producto_id,
    fecha_venta,
    cantidad,
    monto_total,
    'tienda' AS canal,
    creado,
    inserted_at
FROM g0_cmc_ventas.bronze.raw_ventas_tienda
```

Código SQL que permite crear una vista para unir información de dos fuentes distintas

```
%sql
CREATE OR REPLACE MATERIALIZED VIEW g0_cmc_ventas.silver.mv_ventas
(
    cod_mes STRING COMMENT 'Código del mes en que se realizó la venta en formato YYYYMM',
    venta_id STRING COMMENT 'Identificador único de la venta',
    cliente_id STRING COMMENT 'Identificador único del cliente',
    producto_id STRING COMMENT 'Identificador único del producto',
    fecha_venta DATE COMMENT 'Fecha en que se realizó la venta',
    cantidad INTEGER COMMENT 'Cantidad de productos vendidos',
    monto_total DOUBLE COMMENT 'Precio unitario por cada producto',
    canal STRING COMMENT 'Canal por el que se realizó la venta',
    created_at TIMESTAMP COMMENT 'Timestamp en que se creó el registro',
    inserted_at TIMESTAMP COMMENT 'Timestamp en que se actualizó el registro'
)
COMMENT 'Vista materializada que contiene el consolidado de ventas'
AS
SELECT
    DATE_FORMAT(fecha_venta, 'yyyyMM') AS cod_mes,
    venta_id,
    cliente_id,
    producto_id,
    fecha_venta,
    cantidad,
    monto_total,
    canal,
    CAST(creado AS TIMESTAMP) AS created_at,
    CURRENT_TIMESTAMP() AS inserted_at
FROM g0_cmc_ventas.silver.vw_ventas
```

Código SQL que permite crear una vista (materializada) como tabla transformada de Silver



ETAPA 4: Desarrollo en Gold

Catálogo: Ventas

```
%sql
CREATE OR REPLACE VIEW g0_cmc_ventas.gold.vw_ventas
COMMENT 'Esta vista contiene detalles de ventas'
AS
SELECT
  cod_mes,
  venta_id,
  cliente_id,
  producto_id,
  fecha_venta,
  cantidad,
  monto_total,
  canal,
  created_at,
  inserted_at
FROM g0_cmc_ventas.silver.mv_ventas
```

Código SQL que permite crear una vista "vw_ventas", la cual será utilizada por otros dataproducts.

```
%sql
CREATE OR REPLACE VIEW g0_cmc_ventas.gold.vw_ventas_por_producto
(
  cod_mes COMMENT 'View - Código del mes en que se realizó la venta en formato YYYYMM',
  fecha_venta COMMENT 'View - Fecha en que se realizó la venta',
  producto_id COMMENT 'View - Identificador único del producto',
  total_productos COMMENT 'Cantidad de productos vendidos',
  total_monto COMMENT 'Monto total vendido'
)
COMMENT 'Vista que contiene el consolidado de ventas'
AS
SELECT
  cod_mes,
  fecha_venta,
  producto_id,
  SUM(cantidad) AS total_productos,
  SUM(monto_total) AS total_monto
FROM g0_cmc_ventas.gold.vw_ventas
GROUP BY
  cod_mes,
  fecha_venta,
  producto_id
```

Código SQL que permite crear una vista agregada por productos

ETAPA 4: Procesamiento incremental

Catálogo: Clientes

```
%sql
MERGE INTO g0_mkt_clientes.silver.clientes cli
USING g0_mkt_clientes.bronze.raw_clientes raw ON cli.cliente_id = raw.cliente_id
WHEN MATCHED THEN UPDATE SET
    cli.nombre = raw.nombre,
    cli.email = raw.email,
    cli.fecha_nacimiento = raw.fecha_nacimiento,
    cli.genero = raw.genero,
    cli.ciudad = raw.ciudad,
    cli.creado = raw.creado,
    cli.inserted_at = raw.inserted_at
WHEN NOT MATCHED THEN INSERT
    (cliente_id, nombre, email, fecha_nacimiento, genero, ciudad, creado, inserted_at)
VALUES
    (raw.cliente_id, raw.nombre, raw.email, raw.fecha_nacimiento, raw.genero, raw.ciudad, raw.created_at, current_timestamp())
```

Código SQL que permite realizar un procesamiento incremental a partir de la tabla raw clientes.

ETAPA 4: Validaciones y Test de Calidad

Catálogo: Clientes

```

7
Python
spark.sql(f"""
CREATE OR REPLACE TEMPORARY VIEW errores_formato_email AS
SELECT
COUNT(*) AS total_errores,
'errores_formato_email' AS tipo_error,
CAST('{INGESTION_DATE}' AS DATE) as created_at
FROM data_a_validar
WHERE email IS NOT NULL
AND email NOT LIKE '%@%.%'
""")
> See performance (1)
Optimize
DataFrame[]

8
Python
spark.sql(f"""
CREATE OR REPLACE TEMPORARY VIEW errores_email_nulos AS
WITH counts AS (
SELECT
COUNT(*) AS total_count,
COUNT(CASE WHEN email IS NULL THEN 1 END) AS email_null_count
FROM data_a_validar
)
SELECT
COUNT(CASE WHEN email_null_count > (0.1 * total_count) THEN 1 END) AS total_errores,
'errores_email_nulos' AS tipo_error,
CAST('{INGESTION_DATE}' AS DATE) as created_at
FROM counts
""")
> See performance (1)
Optimize

```

```

9
Python
spark.sql(f"""
CREATE OR REPLACE TEMPORARY VIEW errores_validacion AS
SELECT 'clientes_master' AS process, total_errores, tipo_error, created_at FROM errores_formato_email
UNION ALL
SELECT 'clientes_master' AS process, total_errores, tipo_error, created_at FROM errores_email_nulos
""")
> See performance (1)
Optimize
DataFrame[]

10
Python
df_errores = spark.sql(
"""
SELECT COUNT(*) as total_errores
FROM errores_validacion
WHERE total_errores > 0
""")
total_errores = df_errores.collect()[0][0]

if total_errores == 0:
    dbutils.notebook.exit("No se encontraron errores de validación.")
> See performance (1)
Optimize
df_errores: pyspark.sql.connect.dataframe.DataFrame = [total_errores: long]

11
Python
spark.sql(f"""
INSERT INTO g0_mkt_clientes.support.errores_validacion (process, total_errores, tipo_error, created_at)
SELECT process, total_errores, tipo_error, created_at FROM errores_validacion
""")
> See performance (1)
Optimize

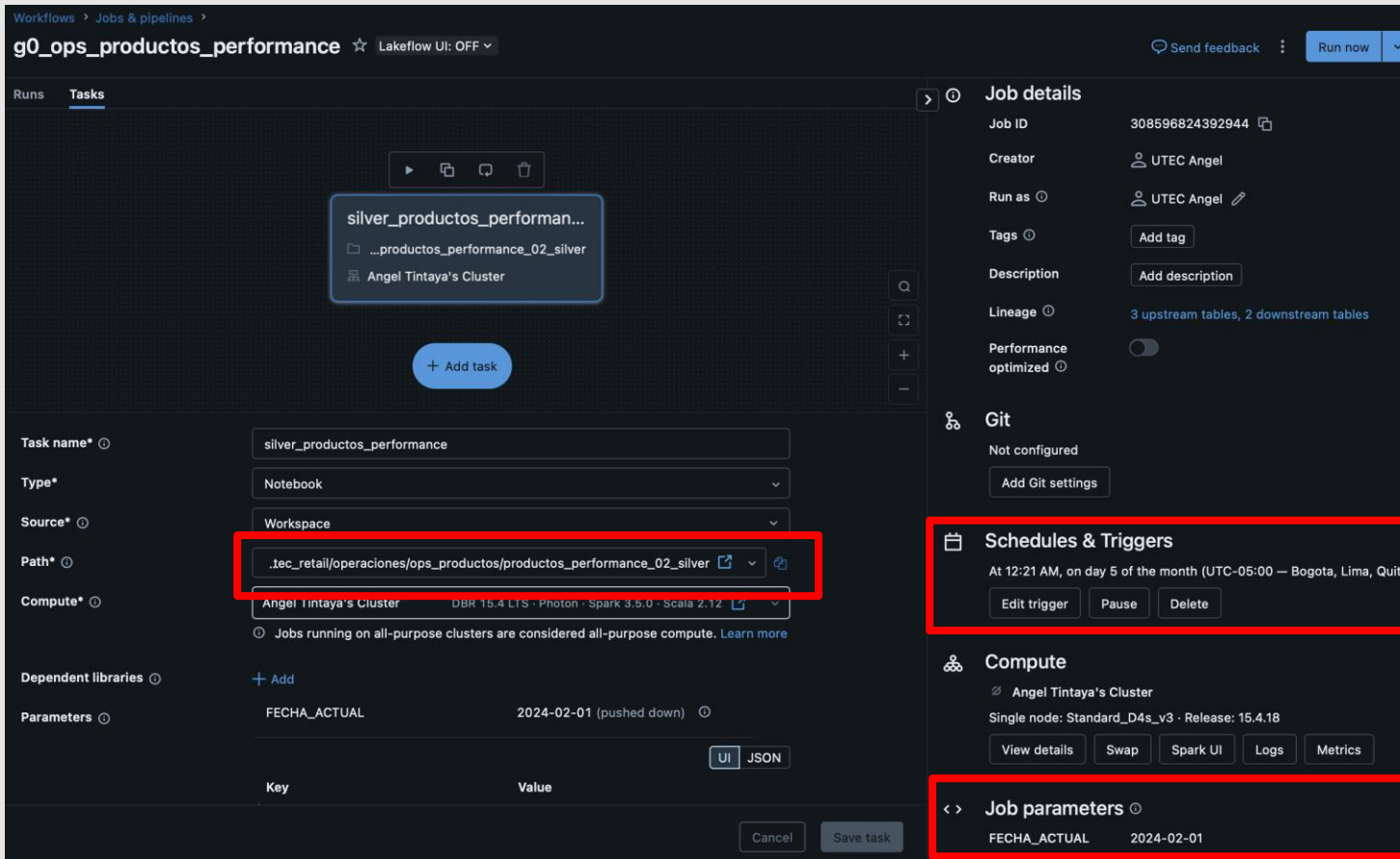
```

Código SQL que permite realizar validaciones e insertar los resultados para control de errores.



ETAPA 4: Programar Workflows o pipelines

Catálogo: Productos



The screenshot displays the Lakeflow interface for configuring a task. The task name is 'silver_productos_performance'. The source is set to 'Workspace'. The path is highlighted with a red box: '.tec_retail/operaciones/ops_productos/productos_performance_02_silver'. The compute resource is 'Angel Tintaya's Cluster'. The dependent libraries section shows 'FECHA_ACTUAL' with a value of '2024-02-01 (pushed down)'. The job parameters section is also highlighted with a red box, showing 'FECHA_ACTUAL' with a value of '2024-02-01'. The right sidebar shows job details, including the job ID, creator, and tags. The 'Schedules & Triggers' section is highlighted with a red box, showing a trigger set to 'At 12:21 AM, on day 5 of the month (UTC-05:00 — Bogota, Lima, Quit.)'. The 'Compute' section is also highlighted with a red box, showing the cluster name and node details.

En el siguiente workflow `g0_ops_productos_performance`, se observa:

- La ruta dónde se encuentra el código asociado al task (**Path**).
- Frecuencia de ejecución (**Schedules & Triggers**)
- Parámetro de ejecución (**Job parameters**)

ETAPA 4: Guardar logs y monitoreos del pipeline

Catálogo: Productos

Go to the latest successful run Cancel runs ▾

Start time	Run ID	Launched	Duration	Spark	Status	Error code	Run parameters	
Jun 27, 2025 at ...	44784585...	Manually	42s	Spark UI / Logs / Metrics	✓ Succe...		FECHA_ACTU...	⋮
Jun 26, 2025 at ...	10725726...	Manually	41s	Spark UI / Logs / Metrics	✓ Succe...		FECHA_ACTU...	⋮
Jun 26, 2025 at ...	81453280...	Manually	1m 16s	Spark UI / Logs / Metrics	✓ Succe...		FECHA_ACTU...	⋮
Jun 26, 2025 at ...	30259324...	Manually	56s	Spark UI / Logs / Metrics	✓ Succe...		FECHA_ACTU...	⋮
Jun 26, 2025 at ...	14568253...	Manually	9m 8s	Spark UI / Logs / Metrics	✗ Failed	RunExecutionE...	FECHA_ACTU...	⋮
Jun 26, 2025 at ...	73729371...	Manually	1m 5s	Spark UI / Logs / Metrics	✗ Failed	RunExecutionE...	FECHA_ACTU...	⋮

Tabla resumen que permite monitorear el estado del pipeline: g0_ops_productos_performance



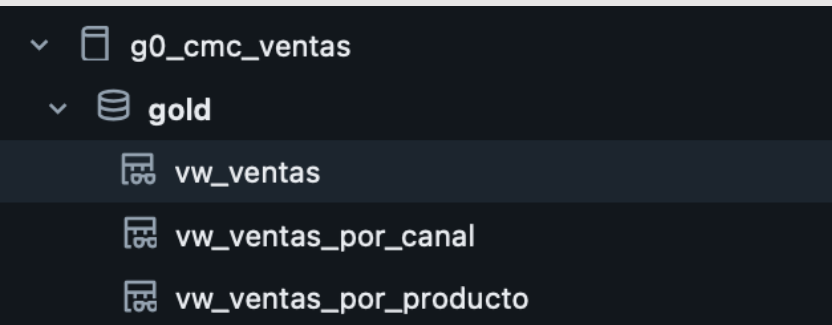
ETAPA 5: Exposición y Governanza

Objetivo: Exponer los datos para su consumo, controlado y documentado.

Pasos	Descripción
Publicar tablas gold con acceso controlado	Gold debe tener solo data lista para consumo
Habilitar Lineage	Mostrar trazabilidad de las tablas
Asegurar acceso gobernado (RBAC)	Grupos con permisos sobre cada schema y tabla
Proteger columnas sensibles (column-level security)	Masking, etiquetas de confidencialidad, accesos restringidos
Habilitar Data Discovery	Permitir a otros equipos descubrir el catálogo desde Data Explorer
Registro de métricas del producto	¿Cómo medimos si este producto es útil o no? (frecuencia de uso, etc.)

ETAPA 5: Tablas gold con acceso controlado

Dominio: Comercial



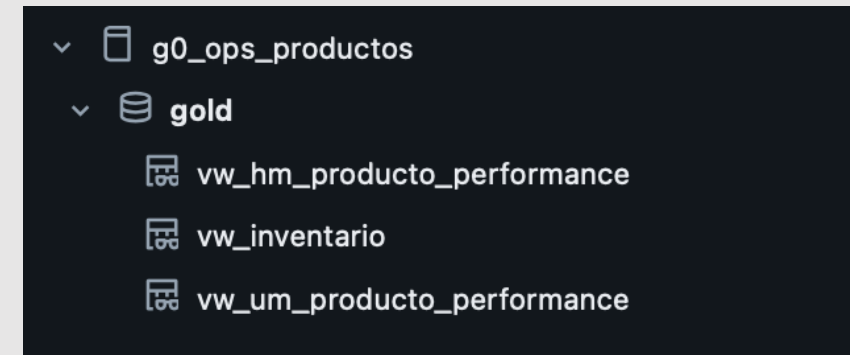
Esquema Gold perteneciente al
Data Product: Ventas

Dominio: Marketing



Esquema Gold perteneciente al
Data Product: Clientes

Dominio: Operaciones

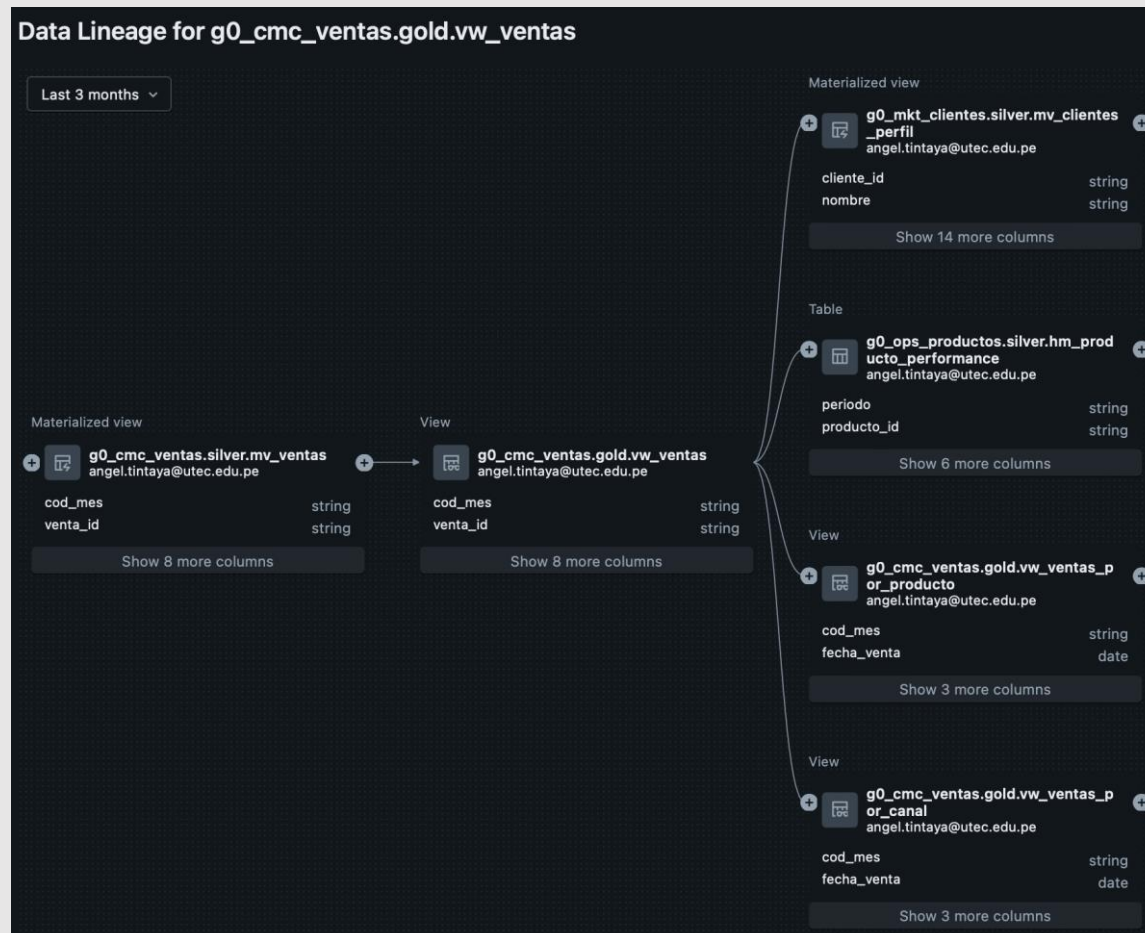


Esquema Gold perteneciente al
Data Product: Productos

Tablas Gold publicadas en Unity Catalog, listas para ser consumidas.



ETAPA 5: Lineage en Databricks



Data Lineage de la vista
g0_cmc_ventas.gold.vw_ventas

Se observa:

- Se creó a partir de:
g0_cmc_ventas.silver.mv_ventas
- Está siendo consumida en los tres dominios: Marketing, Operaciones y Comercial.



ETAPA 5: Asegurar Acceso Gobernado

Workspace settings > Identity and access >

Groups

Filter groups 9 total

Name	Members	Source
admins	3	System ⓘ
g0_mkt_clientes_admin	2	Account ⓘ
g0_mkt_clientes_reader	0	Account ⓘ
g0_mkt_clientes_writer	2	Account ⓘ

Grupos reader, writer y admin asociados al Data Product
Clientes

- Por cada Data Product se crean grupos con privilegios: admin, reader y writer.
- Los accesos al Data Product se deben dar sólo a través de estos grupos.



ETAPA 5: Asegurar Acceso Gobernado

Catalog Explorer >

g0_mkt_clientes  

Overview Details **Permissions** Workspaces

Grant Revoke Privileges g0_mkt_clientes_reader

Principal	Privilege	Object
g0_mkt_clientes_reader	BROWSE	 g0_mkt_clientes
g0_mkt_clientes_reader	EXECUTE	 g0_mkt_clientes
g0_mkt_clientes_reader	READ VOLUME	 g0_mkt_clientes
g0_mkt_clientes_reader	SELECT	 g0_mkt_clientes
g0_mkt_clientes_reader	USE CATALOG	 g0_mkt_clientes
g0_mkt_clientes_reader	USE SCHEMA	 g0_mkt_clientes
All account users	BROWSE	 g0_mkt_clientes

- Grupos reader, writer y admin son asignados al catalog con sus respectivos privilegios.
- Sólo miembros del Dominio de Datos pueden tener acceso writer o admin.
- El acceso reader se brindará a usuarios consumidores del dataproduct (Externos al dominio)

Grupo reader del Data Product Clientes con sus respectivos privilegios.



ETAPA 5: Protección de columnas sensibles

Catalog Explorer > g0_mkt_clientes > gold >

vw_clientes  

Open in a dashboard  Create 

Overview **Sample Data** Details Permissions History Lineage Insights Quality

 Ask Genie Preview Explore data in natural language with Genie AI 

	^A _C cliente_id	^A _C nombre	^A _C email	^A _C fecha_nacimiento	^A _C genero	^A _C ciudad	 created_at
1	C001	Juan Pérez	xxxx.x@xxxxx.xxx	zleSxJkyqy09UYTAZFGQOw...	X	Lima	2025-05-01T00:00:00.000+0
2	C002	María López	xxxxx.x@xxxxxx....	EQautNdWsEtYnAFyb4YIEg==	X	Arequipa	2025-05-01T00:00:00.000+0
3	C003	Luis García	xxxxx@xxxxx.xxx	kDhnZx3iSi7m9d6jWqis3g==	X	Cusco	2025-05-01T00:00:00.000+0
4	C004	Ana Torres	xxx.x@xxxxx.xxx	rfDq5Nr8svmeSTcfhUQmpw...	X	Lima	2025-05-01T00:00:00.000+0

Datos de muestra de tabla Gold del Data Produc Clientes
desde un usuario **SIN** acceso a PII



ETAPA 5: Protección de columnas sensibles

Catalog Explorer > g0_mkt_clientes > gold >

vw_clientes  

Open in a dashboard  Create 

Overview **Sample Data** Details Permissions History Lineage Insights Quality

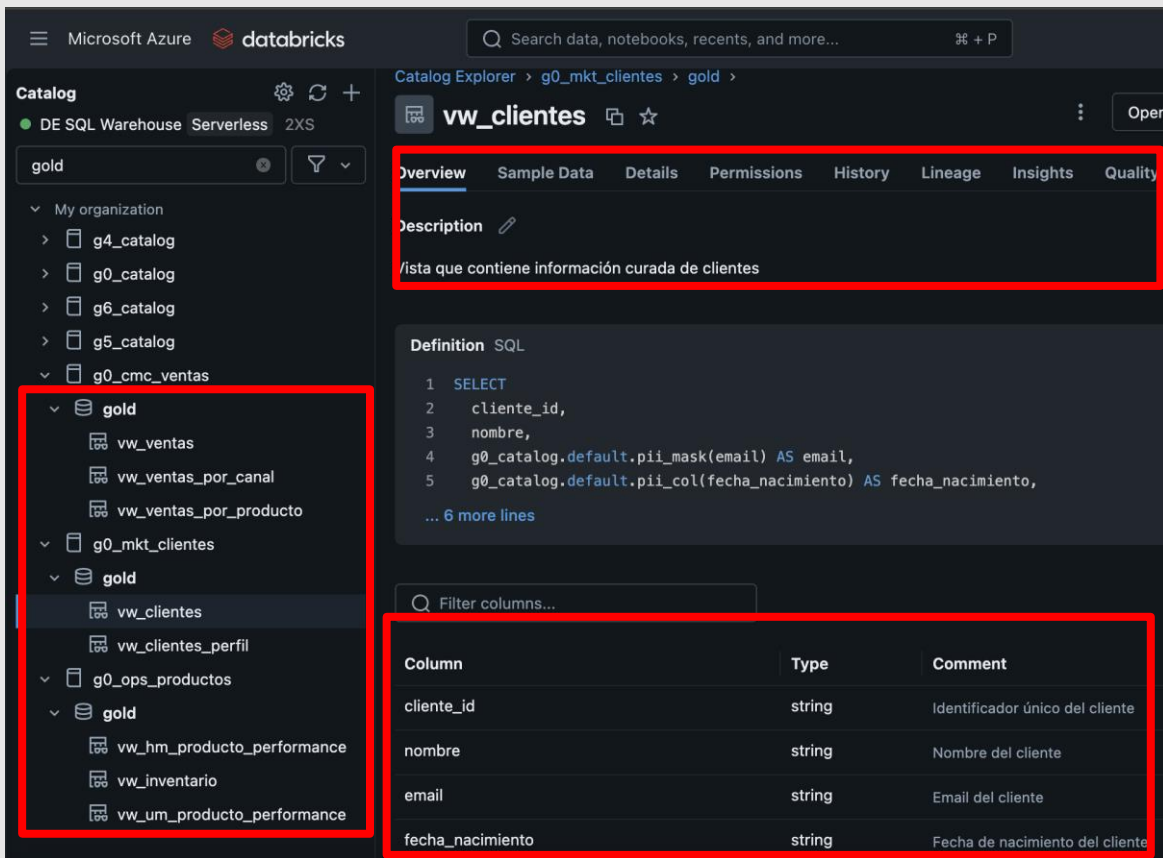
 Ask Genie Preview Explore data in natural language with Genie AI 

	 cliente_id	 nombre	 email	 fecha_nacimiento	 genero	 ciudad	 created_at
1	C001	Juan Pérez	juan.p@gmail.com	1990-03-15	M	Lima	2025-05-01T00:00:00.000+00:00
2	C002	María López	maria.l@hotmail.com	1985-07-22	F	Arequipa	2025-05-01T00:00:00.000+00:00
3	C003	Luis García	luisg@yahoo.com	1978-11-02	M	Cusco	2025-05-01T00:00:00.000+00:00
4	C004	Ana Torres	ana.t@gmail.com	1995-06-18	F	Lima	2025-05-01T00:00:00.000+00:00

Datos de muestra de tabla Gold del Data Product Clientes
desde un usuario **CON** acceso a PII



ETAPA 5: Habilitar Data Discovery



vw_clientes

Description

/ista que contiene información curada de clientes

Definition SQL

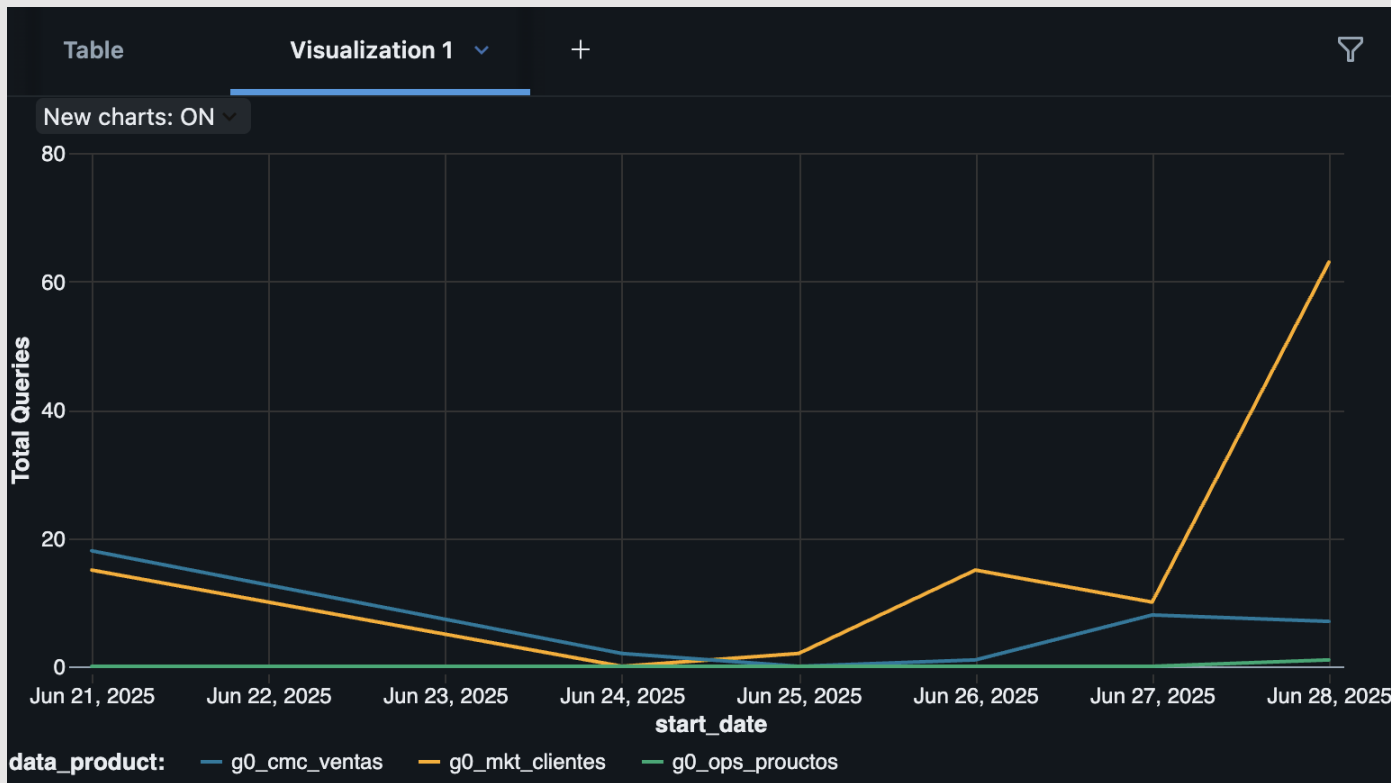
```
1 SELECT
2   cliente_id,
3   nombre,
4   g0_catalog.default.pii_mask(email) AS email,
5   g0_catalog.default.pii_col(fecha_nacimiento) AS fecha_nacimiento,
... 6 more lines
```

Column	Type	Comment
cliente_id	string	Identificador único del cliente
nombre	string	Nombre del cliente
email	string	Email del cliente
fecha_nacimiento	string	Fecha de nacimiento del cliente

Con Unity Catalog podemos:

- Explorar todas las tablas/vistas de los Data Product a los que tenemos acceso.
- Visualizar descripción de las tablas/vistas y de sus columnas.
- De ser una vista, revisar cómo fue creada.
- Ver una muestra de los datos a los que tenemos acceso..

ETAPA 6: Registro de métricas de producto



Total queries por día por Data Product

Es posible obtener métricas para identificar la usabilidad del dataproduct. Por ejemplo:

- Total de consultas.
- Estado de ejecución.
- Duración total de consulta.
- Duración de ejecución.



ETAPA 6: Escalado y federación

Objetivo: Escalar el modelo a más dominios y asegurar la autonomía.

Pasos	Descripción	Ejemplo
Replicar a otros dominios	Extender enfoque a nuevos productos, zonas, áreas.	Automatizaciones del equipo de data platform facilitan la adopción en otros dominios.
Autonomía en cada dominio	Cada equipo es dueño de su ciclo de vida.	Cada owner gestiona sus propios pipelines sin depender de un equipo central.
Evaluación continua del Mesh	Revisar calidad, uso, performance y satisfacción.	Revisión mensual de KPIs de uso y calidad de los data products.

A tener en cuenta ...

- Las Materialized Views (MV) solo pueden ejecutarse en un SQL Warehouse.
- El SQL Warehouse solo permite sentencias SQL (no se puede usar Python ni PySpark).
- Si el pipeline va a crear o reemplazar tablas, asegurarse de ejecutar un DROP antes del primer uso.



A tener en cuenta ...

- Los parámetros variables (como fechas, paths, flags) deben definirse explícitamente en el pipeline.
- Conocer y reutilizar los códigos y convenciones definidos por el equipo de Data Platform.
- Solicitar la creación de un nuevo catalog al equipo de Data Platform.





Takeaways

- La arquitectura Medallion permite orquestar calidad de datos desde la ingesta hasta el consumo.
- La automatización con workflows garantiza consistencia, trazabilidad y entrega continua de valor.
- El descubrimiento de datos mediante catalogación acelera la colaboración entre dominios.
- La gobernanza federada empodera a los equipos de dominio sin perder el control central.
- Diseñar data products con preguntas de negocio claras garantiza alineación entre la solución técnica y las decisiones estratégicas.

¡Gracias!



UTEC Posgrado